

```

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Circle, FancyArrowPatch
import copy

# ===== IMPLEMENTAÇÃO MANUAL DE GAUSS-JORDAN =====
def gauss_jordan(A, b, tol=1e-10):
    """
    Resolve  $Ax = b$  pelo método de Gauss-Jordan (implementação manual).
    Retorna solução x e indica se o sistema é singular.
    """
    n = len(b)
    # Matriz aumentada
    M = np.hstack((A.astype(float), b.reshape(-1, 1)))

    # Eliminação para frente
    for i in range(n):
        # Pivoteamento parcial
        max_row = np.argmax(np.abs(M[i:, i])) + i
        if abs(M[max_row, i]) < tol:
            return None, True # Singular

        # Troca de linhas
        M[[i, max_row]] = M[[max_row, i]]

        # Normaliza pivô
        pivot = M[i, i]
        M[i] = M[i] / pivot

        # Elimina coluna
        for k in range(n):
            if k != i:
                factor = M[k, i]
                M[k] = M[k] - factor * M[i]

    x = M[:, -1]
    return x, False

# ===== CLASSE PRINCIPAL =====
class TrussAnalyzer:
    def __init__(self):

```

```

self.N = 0
self.B = 0
self.coords = None          # (N, 2) array
self.connectivity = None    # (B, 2) array
self.Fx = None
self.Fy = None
self.supports = None        # lista de strings: 'livre', 'movel',
'fixo'

self.forces = None          # forças nas barras (positivo = tração)
self.reactions = None

def setup(self, coords, connectivity, Fx, Fy, supports):
    self.coords = np.array(coords, dtype=float)
    self.connectivity = np.array(connectivity, dtype=int)
    self.Fx = np.array(Fx, dtype=float)
    self.Fy = np.array(Fy, dtype=float)
    self.supports = supports
    self.N = len(coords)
    self.B = len(connectivity)

    # Validações básicas
    assert self.coords.shape == (self.N, 2)
    assert self.connectivity.shape == (self.B, 2)
    assert len(Fx) == len(Fy) == self.N
    assert len(supports) == self.N

def build_equilibrium_matrix(self):
    """Monta matriz de equilíbrio [A] {x} = {F} onde x =
[forças_barras, reações]"""
    # Contagem de reações
    reaction_map = []
    col_idx = self.B        # colunas após as barras

    for node in range(self.N):
        supp = self.supports[node]
        if supp == 'fixo':
            reaction_map.append((node, 0, col_idx))    # Fx
            col_idx += 1
            reaction_map.append((node, 1, col_idx))    # Fy
            col_idx += 1
        elif supp == 'movel':    # assumimos rolete horizontal
(reação vertical)
            reaction_map.append((node, 1, col_idx))    # Fy

```

```

        col_idx += 1
        # livre = nenhuma reação

total_eqs = 2 * self.N
total_vars = self.B + (col_idx - self.B)
A = np.zeros((total_eqs, total_vars))
b = np.zeros(total_eqs)

# Preenche b com cargas (negativo porque move para lado
direito)
for i in range(self.N):
    b[2*i] = -self.Fx[i]
    b[2*i + 1] = -self.Fy[i]

L = np.zeros((self.B, 2)) # vetores direção das barras

for e in range(self.B):
    n1, n2 = self.connectivity[e]
    dx = self.coords[n2, 0] - self.coords[n1, 0]
    dy = self.coords[n2, 1] - self.coords[n1, 1]
    length = np.sqrt(dx**2 + dy**2)
    if length < 1e-8:
        raise ValueError(f"Barra {e} com comprimento zero")
    L[e] = [dx/length, dy/length]

# Preenche matriz A
for e in range(self.B):
    n1, n2 = self.connectivity[e]
    cx, cy = L[e]

    # Nó 1
    A[2*n1, e] = cx # Fx
    A[2*n1 + 1, e] = cy # Fy
    # Nó 2 (sentido oposto)
    A[2*n2, e] = -cx
    A[2*n2 + 1, e] = -cy

# Reações
for node, dof, col in reaction_map:
    row = 2 * node + dof
    A[row, col] = 1.0

return A, b, total_vars, reaction_map

```

```

def solve(self):
    A, b, total_vars, reaction_map =
self.build_equilibrium_matrix()

    x, singular = gauss_jordan(A, b)
    if singular:
        raise ValueError("Sistema singular - treliça não é
isostática ou instável")

    self.forces = x[:self.B]

    # Reações (para estatísticas)
    self.reactions = {}
    for node, dof, col in reaction_map:
        self.reactions[(node, dof)] = x[col]

    return self.forces

def get_bar_stats(self):
    tensions = self.forces[self.forces > 1e-6]
    compressions = self.forces[self.forces < -1e-6]

    stats = {
        'max_tension': tensions.max() if len(tensions) > 0 else 0,
        'max_compression': abs(compressions.min()) if
len(compressions) > 0 else 0,
        'most_loaded_bar': np.argmax(np.abs(self.forces)),
        'mean_force': np.mean(np.abs(self.forces)),
        'sum_reactions': sum(abs(v) for v in
self.reactions.values())
    }
    return stats

# ===== VISUALIZAÇÕES =====
def plot_original(self, title="Treliza Original"):
    fig, ax = plt.subplots(figsize=(10, 8))
    coords = self.coords

    # Barras
    for e in range(self.B):
        n1, n2 = self.connectivity[e]
        x = [coords[n1,0], coords[n2,0]]

```

```

        y = [coords[n1,1], coords[n2,1]]
        ax.plot(x, y, 'k-', linewidth=2)

# Nós
for i in range(self.N):
    color = 'blue' if self.supports[i] == 'fixo' else 'green'
    if self.supports[i] == 'movel' else 'gray'
    size = 80 if self.supports[i] != 'livre' else 60
    ax.scatter(coords[i,0], coords[i,1], s=size, c=color,
zorder=5)
    ax.text(coords[i,0]+0.2, coords[i,1]+0.2, f'{i}',
fontsize=12, fontweight='bold')

ax.set_xlabel('X (m)')
ax.set_ylabel('Y (m)')
ax.set_title(title)
ax.grid(True, alpha=0.3)
ax.axis('equal')
plt.show()

def plot_forces(self, title="Trelia com Forças Internas"):
    fig, ax = plt.subplots(figsize=(12, 9))
    coords = self.coords
    forces = self.forces

    fmax = max(abs(forces).max(), 1e-6)

    for e in range(self.B):
        n1, n2 = self.connectivity[e]
        x = [coords[n1,0], coords[n2,0]]
        y = [coords[n1,1], coords[n2,1]]
        f = forces[e]

        # Cor: vermelho = compressão, azul = tração
        color = 'red' if f < 0 else 'blue'
        width = 1 + 8 * abs(f) / fmax

        ax.plot(x, y, color=color, linewidth=width, alpha=0.85)

    # Texto da força
    mx = (x[0] + x[1]) / 2
    my = (y[0] + y[1]) / 2

```

```

        ax.text(mx, my, f'{f:.1f}', fontsize=9, ha='center',
va='center',
                bbox=dict(facecolor='white', alpha=0.7,
edgecolor='none', pad=1))

# Nós
for i in range(self.N):
    color = 'blue' if self.supports[i] == 'fixo' else 'green'
if self.supports[i] == 'movel' else 'black'
    ax.scatter(coords[i,0], coords[i,1], s=80, c=color,
zorder=5)
    ax.text(coords[i,0]+0.2, coords[i,1]+0.2, f'{i}',
fontsize=11, fontweight='bold')

ax.set_title(title)
ax.set_xlabel('X (m)')
ax.set_ylabel('Y (m)')
ax.grid(True, alpha=0.3)
ax.axis('equal')

# Legenda simples
ax.plot([], [], color='blue', linewidth=4, label='Tração')
ax.plot([], [], color='red', linewidth=4, label='Compressão')
ax.legend()
plt.show()

def plot_bar_chart(self):
    plt.figure(figsize=(12, 6))
    bars = range(self.B)
    forces = self.forces

    colors = ['blue' if f > 0 else 'red' for f in forces]
    plt.bar(bars, forces, color=colors, alpha=0.8)
    plt.axhline(0, color='black', linewidth=0.8)
    plt.xlabel('Número da Barra')
    plt.ylabel('Força (N) - + Tração / - Compressão')
    plt.title('Forças Internas nas Barras')
    plt.grid(axis='y', alpha=0.3)
    plt.show()

def plot_histogram(self):
    plt.figure(figsize=(10, 6))

```

```

plt.hist(self.forces, bins=20, color='purple', alpha=0.7,
edgecolor='black')
plt.xlabel('Força Interna (N)')
plt.ylabel('Frequência')
plt.title('Histograma das Forças Internas')
plt.grid(alpha=0.3)
plt.show()

# ===== TRELIÇA TESTE (Warren 4 nós)
=====
def create_test_truss():
    """Treliza Warren simétrica - 4 nós, 5 barras"""
    # Coordenadas (m)
    coords = [
        [0.0, 0.0],      # 0 - apoio fixo
        [2.0, 0.0],      # 1 - apoio móvel
        [1.0, 1.732],    # 2 - topo (altura  $\sqrt{3}$  para equilátero)
        [1.0, 0.0]       # 3 - meio inferior
    ]

    connectivity = [
        [0, 3], [3, 1],   # inferiores
        [0, 2], [2, 1],   # diagonais
        [3, 2]            # vertical
    ]

    Fx = [0, 0, 0, 0]
    Fy = [0, 0, 0, -1000] # carga vertical no nó 3

    supports = ['fixo', 'movel', 'livre', 'livre']

    return coords, connectivity, Fx, Fy, supports

# ===== EXECUÇÃO =====
if __name__ == "__main__":
    analyzer = TrussAnalyzer()

    # === Teste com treliza padrão ===
    print("=== TRELIÇA TESTE (Warren 4 nós) ===")
    coords, conn, Fx, Fy, supp = create_test_truss()
    analyzer.setup(coords, conn, Fx, Fy, supp)

```

```

forces = analyzer.solve()
print("Forças nas barras (N):")
for i, f in enumerate(forces):
    status = "TRAÇÃO" if f > 0 else "COMPRESSÃO" if f < 0 else
"NULLA"
    print(f"Barra {i}: {f:8.2f} N → {status}")

stats = analyzer.get_bar_stats()
print("\nEstatísticas:")
print(f"Barra mais solicitada: {stats['most_loaded_bar']}
({abs(forces[stats['most_loaded_bar']]):.1f} N)")
print(f"Força média (módulo): {stats['mean_force']:.1f} N")

# Gráficos
analyzer.plot_original("Treliça Original - Teste")
analyzer.plot_forces("Treliça Colorida por Força Interna")
analyzer.plot_bar_chart()
analyzer.plot_histogram()

print("\n Análise concluída com sucesso!")
print("Código pronto para usar com estruturas maiores
(N=8,16,32).")

```


=== TRELIÇA TESTE (Warren 4 nós) ===

Forças nas barras (N):

Barra 0: 288.68 N → TRAÇÃO

Barra 1: 288.68 N → TRAÇÃO

Barra 2: -577.35 N → COMPRESSÃO

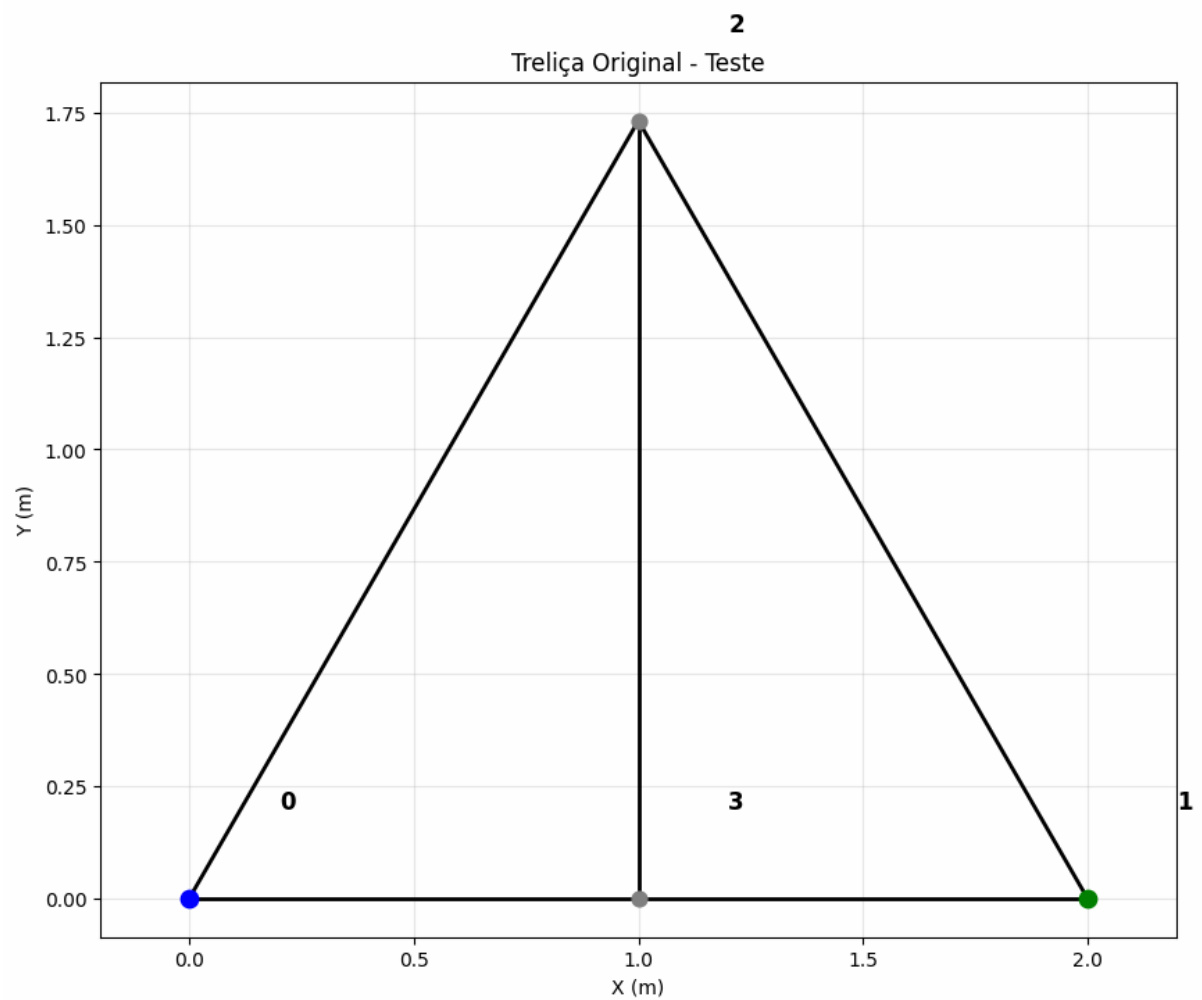
Barra 3: -577.35 N → COMPRESSÃO

Barra 4: 1000.00 N → TRAÇÃO

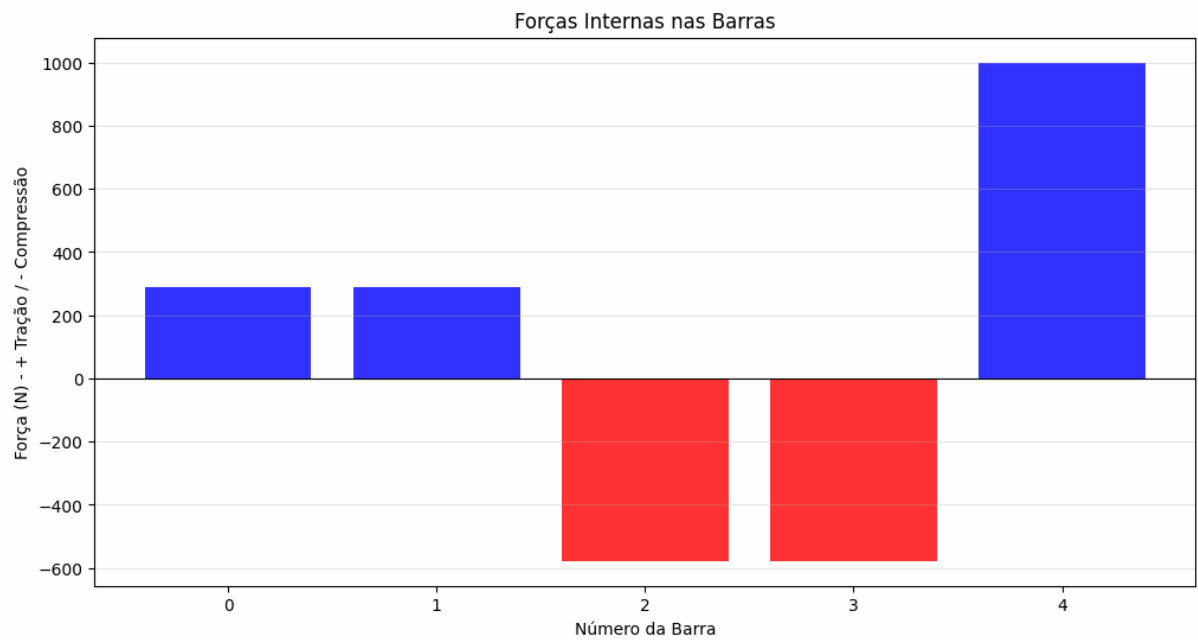
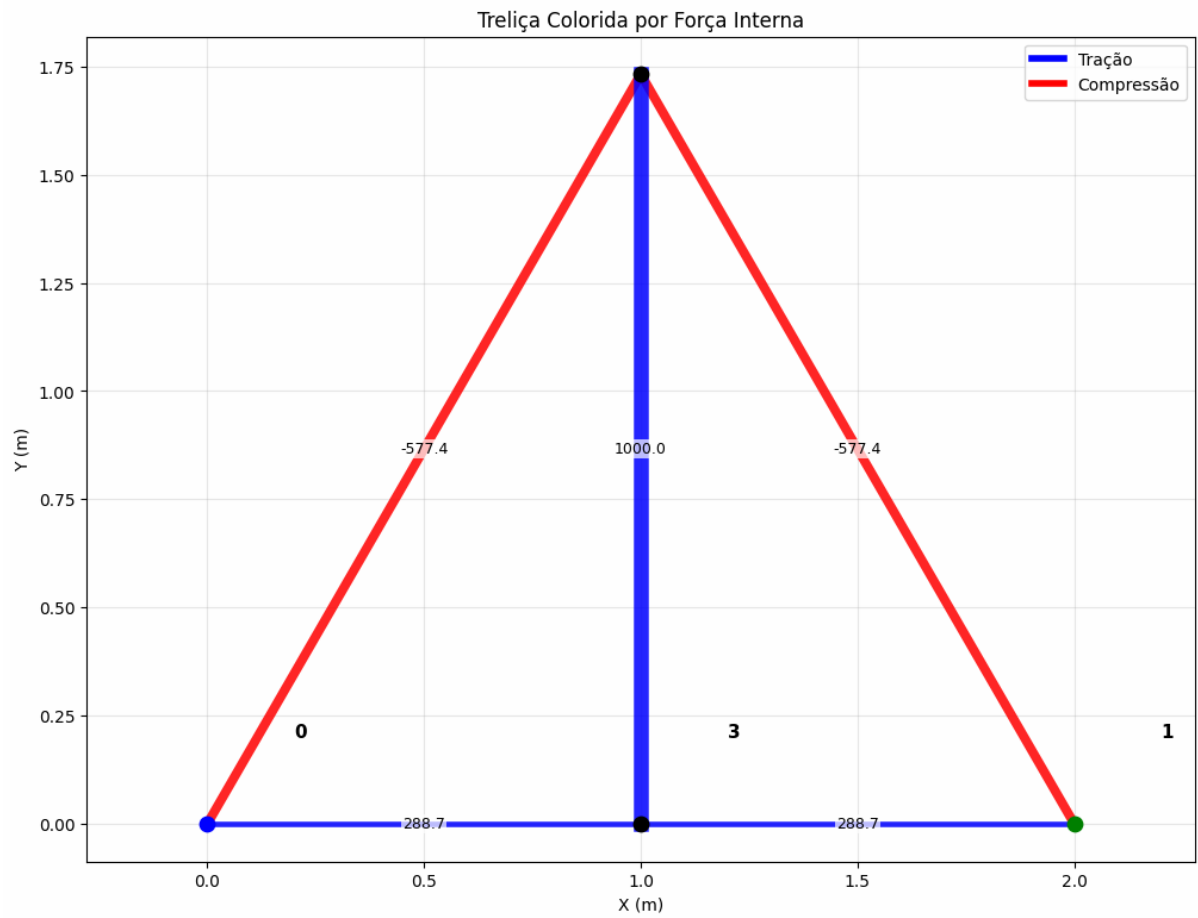
Estatísticas:

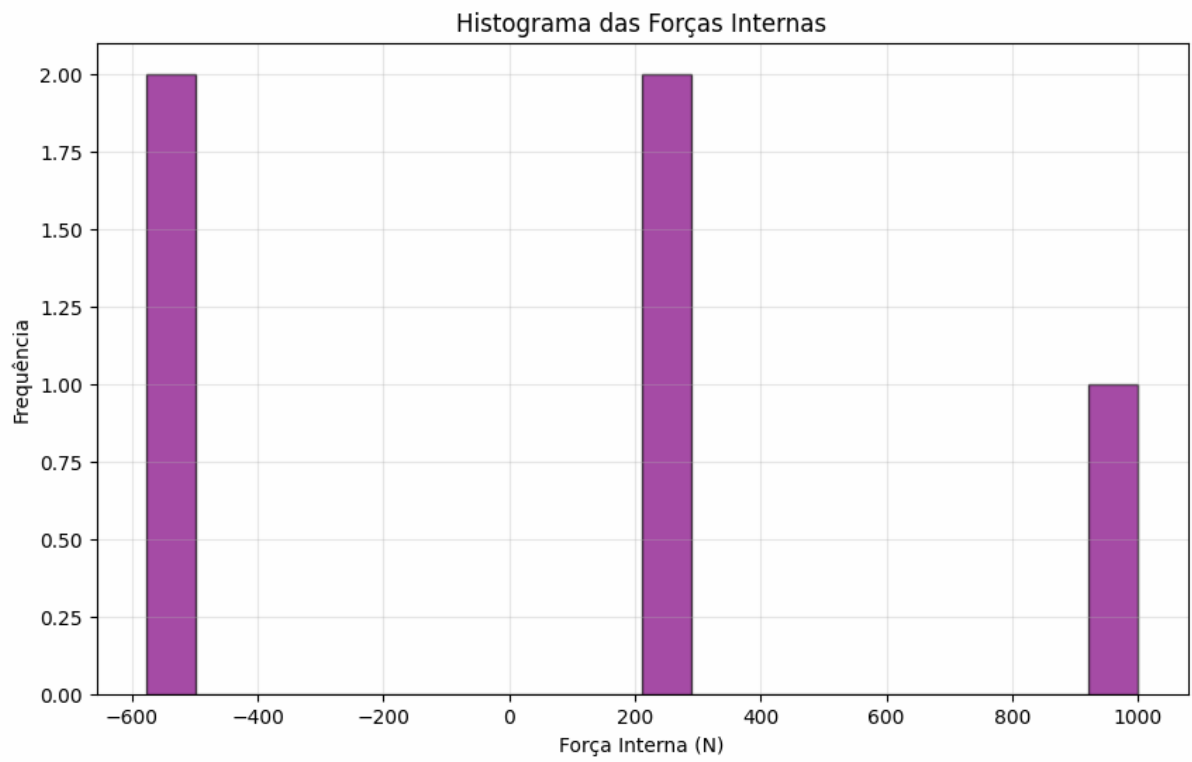
Barra mais solicitada: 4 (1000.0 N)

Força média (módulo): 546.4 N



2





Análise concluída com sucesso!

Código pronto para usar com estruturas maiores (N=8,16,32).